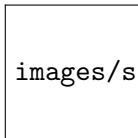


Transistors and Logic Gates

Computer Organization

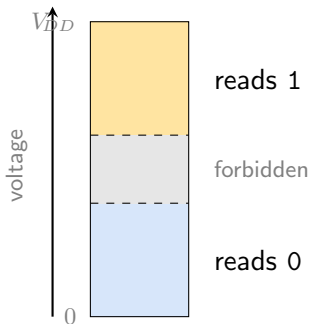
Chandra Gummaluru

`chandra-gummaluru.github.io`



`images/signature.pdf`

A wire carries a continuous voltage, but a digital circuit reads it as only two levels. A voltage near ground is **low** (0) and a voltage near the supply is **high** (1).



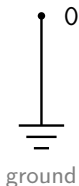
a threshold splits the range in two

voltages in the forbidden band are undefined, so a well-built circuit never rests there

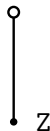
A wire takes its value from whatever it is tied to.



connected to power



connected to ground



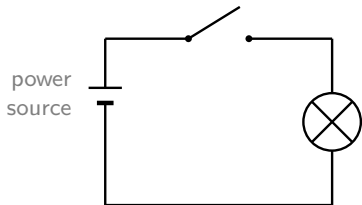
connected to nothing

A wire that is left untied does not carry a 0.

the transistor

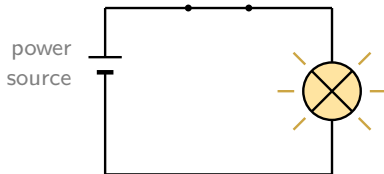
Think of a simple circuit: a power source drives a light bulb through a switch. Current flows only when the switch closes the loop.

open



the switch breaks the loop,
so the bulb stays off

closed



the switch completes the loop,
so the bulb turns on

a hand-flipped switch works for a lamp, but a computer needs a switch that another wire can open and close

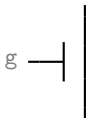
A transistor has three terminals. The voltage on the **gate** decides whether current may flow between the **source** and the **drain**.



the gate acts like the handle of a switch

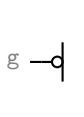
Two kinds of transistor respond to the gate in opposite ways.

nMOS



conducts when the gate
is 1

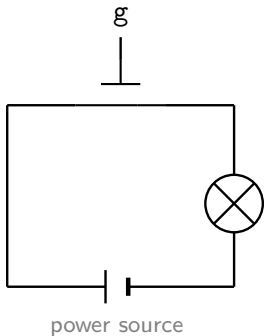
pMOS



conducts when the gate
is 0
(marked by the small circle)

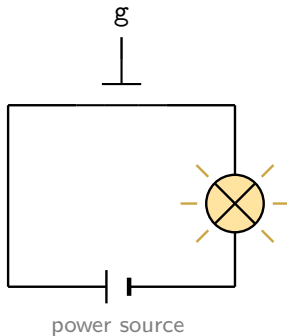
Now the switch is a transistor, sitting where the hand switch was. The gate input g opens and closes it, so a voltage on one wire controls the current in another.

$g = 0$



the gate is low, the nMOS
does not conduct, bulb off

$g = 1$



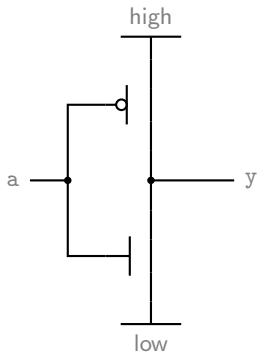
the gate is high, the nMOS
conducts, bulb on

To build a gate, every input must connect the output to either the high supply or the ground.

- a **pull-up** network connects the output to high
- a **pull-down** network connects the output to low

leaving the output unconnected is not the same as driving it low

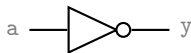
NOT gate



transistor circuit

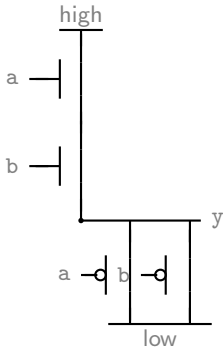
a	y
0	1
1	0

truth table



gate symbol

AND gate



transistor circuit

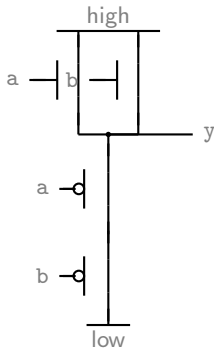
a	b	y
0	0	0
0	1	0
1	0	0
1	1	1

truth table



gate symbol

OR gate



transistor circuit

a	b	y
0	0	0
0	1	1
1	0	1
1	1	1

truth table

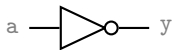


gate symbol

logic gates

From now on we use each gate as a single symbol and describe it by its **truth table**, which lists the output for every input.

we no longer draw the transistors inside

NOT

a	y
0	1
1	0

AND

a	b	y
0	0	0
0	1	0
1	0	0
1	1	1

OR

a	b	y
0	0	0
0	1	1
1	0	1
1	1	1

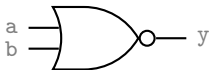
Adding a NOT to the output of AND or OR gives two more gates, drawn with a small circle on the output.

NAND



1 unless both inputs are 1

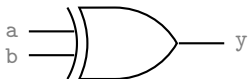
NOR



1 only when both inputs are 0

XOR

output is 1 when the
inputs differ



a	b	y
0	0	0
0	1	1
1	0	1
1	1	0

These few gates are enough to build every circuit in the rest of the course.